

Migración de PostgreSQL 13 a PostgreSQL 18 (mismo servidor)

Sep 30, 2026 · @Kevin

Resumen y alcance

La migración se hace con **pg_dumpall (roles) + pg_dump/pg_restore (cada base)** usando los binarios de PostgreSQL 18, sin parar el servidor 13 durante la copia. Con bases de hasta 50 GB es más simple y seguro que `pg_upgrade`.

Supuestos de esta guía:

- Windows, ambos servidores en la misma máquina, gestionados desde pgAdmin 4.
- PostgreSQL 13 escucha en el puerto `5432` y PostgreSQL 18 en el `5433` (compruébalo en Propiedades > Conexión de cada servidor).
- Binarios de PG 18 en `C:\Program Files\PostgreSQL\18\bin` y carpeta de trabajo `C:\temp`.
- Bases a migrar: `Fortrade`, `Fortrade_test_scripts`, `Test`, `Test4`, `Test 2`, `Test 3`, `fortrade-accountancy` y `fortrade-commons`. La base `postgres` no se migra.
- Se conservan esquemas, owners, permisos (GRANT) y los 22 roles del servidor 13.

Requisitos previos

Antes de tocar nada, completa esta lista:

- ☐ **Backup de seguridad** de las bases en PG 13 (los dumps del paso 2 sirven, pero guarda una copia fuera de `C:\temp`).
- ☐ **Espacio en disco:** los dumps ocupan entre el 20 y el 40 % del tamaño de las bases, más el tamaño final de las bases en el clúster 18. Con 50 GB en total, deja al menos 20 GB libres en `C:\temp` y el doble del tamaño de las bases en la unidad de datos de PG 18.
- ☐ **Extensiones:** lista las de PG 13 y confirma que existen para PG 18 (ver consulta abajo). Instálalas en PG 18 antes de restaurar.
- ☐ **Tablespaces propios:** si hay alguno, crea las carpetas en PG 18 o edita las rutas en `globals.sql`.
- ☐ **Acceso:** usuario `postgres` (o superusuario) y contraseña de ambos servidores. Para no escribirla varias veces, define `$env:PGPASSWORD` en la sesión de PowerShell o usa un archivo `pgpass.conf`.

- ☐ **Versiones de binarios:** usa siempre `pg_dump`, `pg_dumpall` y `pg_restore` de la carpeta de PG 18.

Para listar las extensiones de una base en PG 13 (ejecútalo en cada base):

```
SELECT extname, extversion FROM pg_extension ORDER BY 1;
```

Y para ver cuáles están disponibles en PG 18:

```
SELECT name, default_version FROM pg_available_extensions ORDER BY 1;
```

Paso 1: migrar roles y permisos globales

Los roles no viajan dentro del dump de cada base, por eso se exportan aparte. Esto copia los 22 roles, sus contraseñas (hashes), membresías y opciones, y los tablespaces.

```
cd "C:\Program Files\PostgreSQL\18\bin"
mkdir C:\temp -ErrorAction SilentlyContinue

# 1. Exportar roles desde PG 13 (puerto 5432)
.\pg_dumpall.exe -p 5432 -U postgres --globals-only -f C:\temp\globals.sql

# 2. Revisar el archivo (tablespaces, rol postgres) antes de aplicarlo
notepad C:\temp\globals.sql

# 3. Importar en PG 18 (puerto 5433)
.\psql.exe -p 5433 -U postgres -f C:\temp\globals.sql
```

Qué esperar:

- Errores del tipo `role "postgres" already exists` son normales: ignóralos.
- Si usas tablespaces propios, el archivo contiene sentencias `CREATE TABLESPACE` con rutas de PG 13; ajústalas o elimínalas antes de aplicarlo.
- Si `pg_dumpall` dice que no encuentra `pg_dump`, lanza el comando desde la carpeta `bin` de PG 18, como arriba.

Paso 2: copiar cada base (dump y restore en paralelo)

El formato directorio (`-Fd`) permite paralelizar con `-j`. La opción `-C` incluye el `CREATE DATABASE` con su owner y la configuración a nivel de base, y el dump lleva esquemas, objetos, owners y GRANTS. Ajusta `-j` al número de núcleos de tu CPU (4 es un valor razonable).

Guarda este script como `C:\temp\migrar.ps1` y ejecútalo desde PowerShell:

```

$bin    = "C:\Program Files\PostgreSQL\18\bin"
$work    = "C:\temp"
$jobs    = 4
$dbfs    = "Fortrade","Fortrade_test_scripts","Test","Test4","Test 2","Test
3","fortrade-accountancy","fortrade-commons"

foreach ($d in $dbfs) {
    $dir = Join-Path $work "dump_$d"
    Write-Host "=== $d: dump ==="
    & "$bin\pg_dump.exe" -p 5432 -U postgres -Fd -j $jobs -C -d "$d" -f "$dir"
    if ($LASTEXITCODE -ne 0) { Write-Host "FALLO en dump de $d"; continue }

    Write-Host "=== $d: restore ==="
    & "$bin\pg_restore.exe" -p 5433 -U postgres -j $jobs -C -d postgres "$dir"
    if ($LASTEXITCODE -ne 0) { Write-Host "REVISAR restore de $d" }
}

```

Notas importantes:

- El restore conecta a la base `postgres` de PG 18 y crea cada base nueva desde ahí; no crees las bases a mano antes.
- Si una base ya existe en PG 18 y quieres rehacerla, bórrala primero (`DROP DATABASE "nombre";`) o añade `--clean --if-exists` al `pg_restore` .
- Los nombres con mayúsculas o espacios (`Fortrade` , `Test 2`) se conservan tal cual; por eso van siempre entre comillas.
- Si hay poco espacio, añade `Remove-Item -Recurse -Force $dir` tras un restore correcto para borrar cada dump al terminar.
- Un aviso de `pg_restore` sobre un rol o extensión que ya existe no detiene la copia; un error real en tablas o datos sí conviene revisarlo.

Paso 3: ajustes de rendimiento antes y después

Las estadísticas del planificador no se copian con el dump, así que el análisis posterior es obligatorio. Los ajustes temporales aceleran la creación de índices.

Antes del restore (solo en PG 18, desde psql o pgAdmin):

```

ALTER SYSTEM SET maintenance_work_mem = '1GB';
ALTER SYSTEM SET max_wal_size = '8GB';
SELECT pg_reload_conf();

```

Después del restore:

```
# Recalcular estadísticas en todas las bases de PG 18
& "C:\Program Files\PostgreSQL\18\bin\vacuumdb.exe" -p 5433 -U postgres --all
--analyze-only
```

```
-- Volver a los valores originales
ALTER SYSTEM RESET maintenance_work_mem;
ALTER SYSTEM RESET max_wal_size;
SELECT pg_reload_conf();
```

Ajusta **1GB** y **8GB** a la memoria y el disco disponibles en tu máquina. No los dejes permanentes si el equipo tiene poca RAM.

Verificación

Compara PG 13 (puerto 5432) y PG 18 (puerto 5433) para cada base. Ejecuta cada consulta en los dos servidores y contrasta los resultados.

Qué comprobar	Consulta o comando
Esquemas y permisos de esquema	<code>\dn+</code> en psql
Permisos de tablas	<code>\dp</code> en psql
Owners de las bases	<code>SELECT datname, pg_get_userbyid(datdba) FROM pg_database;</code>
Roles migrados	<code>SELECT count(*) FROM pg_roles;</code>
Extensiones	<code>SELECT extname, extversion FROM pg_extension;</code>
Número de tablas	<code>SELECT count(*) FROM information_schema.tables WHERE table_schema NOT IN ('pg_catalog','information_schema');</code>
Filas por tabla (aprox.)	<code>SELECT relname, n_live_tup FROM pg_stat_user_tables ORDER BY 1;</code>
Tamaño de la base	<code>SELECT pg_size_pretty(pg_database_size(current_data base()));</code>

Para un conteo exacto de las tablas críticas, usa `SELECT count(*) FROM esquema.tabla;` en ambos servidores. El tamaño en PG 18 puede diferir algo del de PG 13 (menos bloat tras el restore); es normal.

Autenticación (`pg_hba.conf`) y contraseñas: PG 18 usa SCRAM-SHA-256 por defecto. Los hashes `md5` migrados siguen funcionando, pero revisa que `pg_hba.conf` del clúster 18 permita a tus aplicaciones conectar con el método correcto. Para pasar un rol a SCRAM, restablece su contraseña con `ALTER ROLE nombre PASSWORD 'nueva';`.

Puesta en producción y rollback

PG 13 no se modifica durante la copia, así que volver atrás es tan simple como seguir usándolo. Para evitar perder datos, congela las escrituras antes del dump definitivo.

1. Ensaya todo el proceso una vez con los dumps y anota cuánto tarda.
2. El día del corte, detén las aplicaciones (o ponlas en solo lectura) para que no haya escrituras en PG 13.
3. Repite el paso 2 con los datos finales (borra antes las bases de prueba en PG 18 con `DROP DATABASE`).
4. Ejecuta el paso 3 (`vacuumdb --analyze-only`) y la verificación.
5. Cambia las cadenas de conexión de las aplicaciones al puerto `5433` . Alternativa: cambia `port` en `postgresql.conf` (PG 18 a `5432` , PG 13 a otro puerto) y reinicia ambos servicios.
6. Prueba las aplicaciones con datos reales y vigila los logs de PG 18 las primeras horas.

Rollback: si algo falla, apunta de nuevo las aplicaciones a PG 13. Como las escrituras posteriores al corte solo existen en PG 18, decide antes del corte cuánto tiempo mantendrás PG 13 intacto (por ejemplo, una o dos semanas) y no lo desinstales hasta entonces.

Si necesitas reducir el tiempo de parada, la alternativa es la **replicación lógica** de PG 13 a PG 18: copia los datos en caliente y el corte dura solo unos minutos, pero exige que las tablas tengan clave primaria y no copia el esquema ni los roles.

Problemas frecuentes y checklist final

Síntoma	Causa probable	Solución
<code>role "xxx" does not exist</code> al restaurar	No se aplicó <code>globals.sql</code>	Ejecuta el paso 1 antes de restaurar
<code>could not open extension control file</code>	Extensión no instalada en PG 18	Instala la extensión (paquete del instalador) y repite el restore de esa base

Síntoma	Causa probable	Solución
<code>database "xxx" already exists</code>	Restore previo incompleto	<code>DROP DATABASE "xxx";</code> y repite
<code>password authentication failed</code>	Método de autenticación o hash distinto	Revisa <code>pg_hba.conf</code> y restablece la contraseña del rol
<code>permission denied for tablespace</code>	Ruta de tablespace inexistente	Crea la carpeta o ajusta <code>globals.sql</code>
Consultas lentas tras migrar	Faltan estadísticas	Ejecuta <code>vacuumdb --all --analyze-only</code>
<code>pg_dump: error: aborting because of server version mismatch</code>	Se usó un <code>pg_dump</code> más antiguo que el servidor	Usa siempre los binarios de PG 18

Checklist final:

- ☐ Backup de PG 13 guardado fuera de `C:\temp`
- ☐ Roles importados (`globals.sql`)
- ☐ Extensiones instaladas en PG 18
- ☐ Las 8 bases restauradas sin errores
- ☐ `vacuumdb --analyze-only` ejecutado
- ☐ Esquemas, owners, permisos y conteos verificados
- ☐ `maintenance_work_mem` y `max_wal_size` restaurados
- ☐ Aplicaciones apuntando a PG 18 y probadas
- ☐ PG 13 conservado hasta cerrar el periodo de rollback